

Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi^{1*}

^{1*}Division of Computer Science, Department of Applied Mathematics,
Faculty of Mathematical Sciences, Ferdowsi University of Mashhad,
Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN,
ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Accurate traffic forecasting remains challenging due to complex spatial and temporal dependencies. Existing graph-based methods, such as Graph Convolutional Networks (GCN) and Temporal GCN (T-GCN), typically rely on predefined adjacency matrices based on physical proximity. However, physical connections often fail to reflect true functional traffic dependencies. To address this limitation, we propose a Graph Structure Learning (GSL) framework that infers the underlying graph structure directly from traffic data using continuous optimization. We investigate two variants: a directed acyclic graph (GSL) and a symmetrized cyclic graph (cGSL), integrated into GCN and T-GCN backbones. Extensive experiments on real-world datasets (SZ-Taxi and Los-loop) demonstrate that replacing static proximity graphs with learned structures significantly improves performance, achieving up to 21.6% and 24.7% reduction in RMSE for GCN and T-GCN baselines, respectively. Notably, we find that spatial-only models benefit more from cyclic graphs, while temporal models perform best with acyclic, directionally learned structures. These findings establish data-driven structure learning as a computationally efficient and interpretable alternative to heuristic graphs, offering urban planners explicit insights into hidden traffic patterns. The paper code is available at <https://github.com/mamintoosi/TGCN-GSL-PyTorch>.

Keywords: Traffic Forecasting, Graph Knowledge Discovery, Graph Structure Learning, Causal Graph Learning, Urban Traffic Mining, Graph Convolutional Networks

1 Introduction

Recent advances in graph knowledge discovery have opened new possibilities for understanding complex urban systems, particularly in transportation networks. This paper applies the principles of graph knowledge discovery to the critical challenge of traffic prediction, demonstrating that uncovering hidden patterns in network connectivity can significantly improve forecasting accuracy. In recent years, the issue of traffic prediction has garnered significant attention in smart urban transportation systems. In the realm of intelligent transportation management and tools such as navigation, ride-hailing, and route planning, accurate traffic prediction plays a crucial role in reducing congestion and increasing the efficiency of transportation networks. Traffic prediction involves analyzing traffic conditions, including flow, speed, extracting traffic patterns, and forecasting traffic trends in urban roads. Due to the complex spatial and temporal dependencies involved in traffic prediction, solving this problem is notoriously challenging and remains a subject of ongoing research.

The spatial dependencies in this problem arise from the fact that changes in traffic volume are influenced by the topological structure of the urban road network; the traffic status in downstream roads is affected by the traffic volume at upstream nodes in the network, and the traffic status at upstream nodes is, in turn, influenced by feedback from downstream nodes. Furthermore, traffic volume dynamically changes over time, affected by short-term and long-term traffic patterns, which underscores the temporal dependencies in traffic data.

In essence, the intricate relationships between roads, traffic patterns, and temporal factors necessitate the development of sophisticated models capable of capturing these complex dependencies. A critical limitation of existing approaches is their reliance on physical proximity or predefined road network topology to construct the graph structure. However, the actual traffic dependencies between roads may not align with their physical relationships - roads that are physically distant might have strong traffic correlations due to commuting patterns, while adjacent roads might have minimal traffic influence on each other.

Multiple traffic prediction methods, including neural networks, statistical approaches, and Bayesian methods, exist, which consider the dynamic change of traffic status over time but neglect spatial dependencies [1]. To describe spatial features, some studies have utilized graph-based methods, such as [2–5]. In [6], a Temporal Graph Convolutional Network (T-GCN) was proposed for traffic prediction. This approach combines a Graph Convolutional Network (GCN) and a Gated Recurrent Unit (GRU). The GCN learns spatial relationships, while the GRU learns temporal dependencies. The combined model can consider both spatial and temporal data, achieving better results than many existing methods in this field.

Existing graph-based methods for traffic prediction rely heavily on predefined spatial relationships, typically constructing the adjacency matrix based on road proximity or distance [6]. While such approaches capture geographical connectivity, they *fail to account for hidden functional dependencies*; where traffic on one road may influence another even if they are not spatially close.

Attention-based methods [7–9] attempt to address this by dynamically adjusting relationship weights between nodes. However, these approaches operate on a fundamental distinction: they learn *implicit relationship weights* rather than discovering the *explicit graph structure* itself. While attention mechanisms can indicate that node j is important to node i at a given time step, they do not reveal whether a persistent, causal dependency exists between them. Consequently, these methods still depend on an initial (often suboptimal) graph topology defined by heuristics or physical proximity, and may therefore miss critical non-local traffic interactions that lack explicit spatial connections.

Our work advances graph knowledge discovery in traffic forecasting by introducing a graph structure learning (GSL) approach that directly infers the adjacency matrix from traffic data. This distinction is fundamental: rather than weighting relationships within a predefined structure, our method discovers the structure itself. The resulting graph represents *explicit, persistent dependencies* between roads; if road A’s traffic consistently influences road B’s future state regardless of physical distance, our learned graph captures this as meaningful, interpretable knowledge. Conversely, it automatically prunes connections between nearby roads that exhibit minimal interaction, revealing that physical adjacency does not guarantee functional dependence.

While graph structure learning has been explored in general graph research [10], this work represents its first application as a knowledge discovery tool for GCN-based traffic forecasting. The learned graph can be analyzed independently as a discovered knowledge graph, offering urban planners and transportation researchers interpretable insights into the hidden causal structure of traffic networks. By shifting from heuristic graph construction to data-driven knowledge extraction, we achieve both more accurate modeling of real-world traffic dynamics and a deeper understanding of the underlying dependencies that drive those dynamics. Our experimental results demonstrate significant error reduction compared to traditional distance-based graphs, validating that explicit structure discovery outperforms implicit relationship weighting.

This finding has broader implications: although many contemporary models achieve high accuracy through complex attention or transformer architectures, their reliance on a predefined or heuristically constructed graph structure remains a fundamental limitation. In this work, we isolate and quantify this limitation by demonstrating that upgrading the graph component of even simple, foundational models (GCN, T-GCN) with a learned structure yields significant performance gains. This suggests that GSL is a powerful, plug-and-play module that could benefit any graph-based traffic forecasting architecture.

In the following sections, we first establish the necessary background by reviewing traffic prediction approaches and Graph Convolutional Networks (Section 2). We then present our key methodological contribution: a novel framework that replaces the conventional distance-based adjacency matrix with a learned graph structure (Section 3). Specifically, Section 3 details:

- Our graph structure learning (GSL) approach for estimating traffic dependencies

- The integration of this learned graph with temporal modeling in the GCN [11] and T-GCN [6] frameworks

We validate our approach through comprehensive experiments (Section 4), demonstrating significant improvements over baseline methods. Finally, we discuss broader implications and future research directions (Section 6), including an analysis of graph acyclicity considerations (Section 5).

2 Background

The goal of traffic forecasting is to predict future traffic conditions based on historical data. Traffic conditions can include metrics like speed, flow, or density - in our experiments, we focus on traffic speed without loss of generality.

We model the road network as an unweighted graph $G = (V, E)$, where:

- $V = \{v_1, \dots, v_N\}$ represents the set of N road segments as nodes
- E represents the set of edges between connected roads
- $\mathbf{A} \in \{0, 1\}^{N \times N}$ is the binary adjacency matrix encoding road connectivity

The traffic data is represented by a feature matrix $X \in \mathbb{R}^{N \times P}$, where:

- N is the number of road segments
- P is the length of the historical time series
- $X_t \in \mathbb{R}^N$ represents the traffic speeds across all roads at time t

The traffic forecasting problem can then be formulated as learning a mapping function f that predicts future traffic conditions:

$$[X_{t+1}, \dots, X_{t+T}] = f(G; [X_{t-n}, \dots, X_t]) \quad (2.1)$$

where n is the number of historical time steps used for prediction and T is the prediction horizon.

2.1 Overview of Traffic Prediction Approaches

Traffic prediction methods can be broadly categorized into several classes:

- **Classical Prediction Methods:** Traditional statistical and machine learning approaches including:
 - Autoregressive Integrated Moving Average (ARIMA) models [12, 13]
 - Support Vector Regression (SVR) [14]
 - Seasonal ARIMA (SARIMA) variants [15]
 - Kalman filtering approaches [16]

These methods, while effective for basic temporal patterns and individual predictions, often struggle with complex spatial dependencies and nonlinear relationships in traffic networks.

- **Classical Deep Learning Methods:**

- Feedforward Neural Networks (FNN) [17]
- Spatio-Temporal Residual Networks (ST-ResNet) [18]
- Interactive Temporal Recurrent Convolutional Networks (ITRCN) [19]
- Recurrent Neural Networks (RNN) and variants: Gated Recurrent Units (GRU) [19], Long Short-Term Memory (LSTM) networks [20, 21]

These methods capture temporal patterns and some spatial relationships, but are limited by their Euclidean assumptions and inability to model complex network topology.

- **Graph-Based and Temporal Graph Methods:**

- Graph Convolutional Networks (GCN) [11, 22]
- Temporal Graph Convolutional Networks (T-GCN) [6]
- Attention-based Temporal Graph Convolutional Networks (A3T-GCN) [7]
- Dynamic Spatial-Temporal Attention Networks [8]
- Spatial-Temporal Similarity Fusion Graph Adversarial Convolutional Networks [23]
- Spatial-Temporal Graph Convolutional Recurrent Networks [24]
- Long-term Spatio-Temporal Graph Attention Network [25]
- Meta Attentive Graph Convolutional Recurrent Network [26]
- Spatio-Temporal Graph Transformer for Dynamic Traffic Flow Prediction [27]
- Trend-aware Attention Spatio-Temporal Graph Convolutional Network [9]

These methods explicitly model network topology and temporal dependencies, but typically rely on predefined graph structures, which may not capture the true underlying relationships in the data.

While comprehensive literature reviews are available [25, 28, 29], our analysis specifically concentrates on the emerging role of graph-based approaches in traffic forecasting. To contextualize this focus, we conducted a systematic bibliometric analysis of 784 articles published between 2000 and 2025, which confirms a significant recent shift towards graph-based methods (see Appendix A for methodology).

Since this paper builds upon GCNs and T-GCN, we next provide a concise overview of graph convolutional networks and the T-GCN method.

2.2 Graph Convolutional Network

Graph Convolutional Networks (GCNs) represent a powerful framework for processing graph-structured data by extending traditional convolutional operations to handle arbitrary network topologies and unordered structures [11, 30]. Through graph pooling operations, GCNs can effectively learn hierarchical abstract representations of input graphs. The versatility of GCNs has led to their successful application across diverse domains, including community detection [31, 32], semi-supervised classification [11, 33], and recommender systems [34]. At their core, GCNs leverage multiple layers of graph convolution operations combined with linear transformations and activation functions to learn sophisticated node-level representations that capture both local and global graph structure.

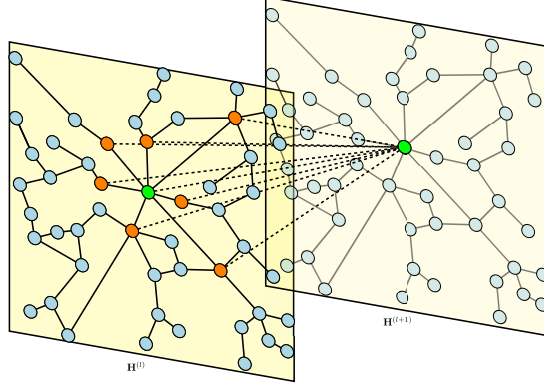


Fig. 1: A graph convolutional neural network architecture showing layers l and $(l+1)$ for computing $\mathbf{H}^{(l+1)}$ as defined in equation (2.2). The green node represents the target node, and its neighboring nodes are shown in orange, corresponding to the connections defined by adjacency matrix $\mathbf{A}$. While traditional methods derive this adjacency matrix from physical road network data, our proposed approach learns the node adjacencies directly from traffic data.

The layer-wise propagation rule in a multi-layer Graph Convolutional Network defines how information flows through successive layers of the network. For layer l , the propagation rule is formulated as follows [11]:

$$\mathbf{H}^{(l+1)} = \sigma\left(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right) \quad (2.2)$$

Here, the key components are:

- $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ represents the adjacency matrix of the undirected graph $\mathcal{G}$ augmented with self-connections, where $\mathbf{A}$ is the original adjacency matrix and $\mathbf{I}_N$ is the $N \times N$ identity matrix
- $\tilde{\mathbf{D}}$ is the degree matrix of $\tilde{\mathbf{A}}$, with diagonal elements defined as: $\tilde{D}_{ii} = \sum_{j=1}^N \tilde{A}_{ij}$
- $\mathbf{W}^{(l)} \in \mathbb{R}^{D \times F}$ is a layer-specific trainable weight matrix that transforms the feature dimension from D to F
- $\sigma(\cdot)$ denotes a non-linear activation function, typically the ReLU function: $\text{ReLU}(x) = \max(0, x)$
- $\mathbf{H}^{(l)} \in \mathbb{R}^{N \times D}$ represents the matrix of node features (activations) in the l^{th} layer, with $\mathbf{H}^{(0)} = \mathbf{X}$ being the input feature matrix

The normalized adjacency term $\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$ performs graph convolution by aggregating and normalizing information from neighboring nodes, while preserving the scale of the features across the network. Hence a two layer GCN can be written as follows:

$$f(\mathbf{A}, \mathbf{X}) = \mathbf{H}^{(2)} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(1)}\mathbf{W}^{(1)}\right) \quad (2.3)$$

The GCN model acts on the nodes of a graph and their first-order neighborhoods to capture the spatial features between the nodes. As shown in Figure 1, the GCN model can be used to obtain the topological relationship between a central node (represented by the green node) and its surrounding nodes (represented by the orange nodes). This allows the GCN model to encode the topological structure of the road network and the attributes on the roads, ultimately obtaining the spatial dependence within the urban transportation graph.

The graph convolutional neural network architecture illustrated in Figure 1 can be applied to such urban transportation graphs for traffic prediction tasks. The green node represents the target node whose future state is being predicted, and the orange nodes represent its neighboring nodes that influence the target node’s future state. By utilizing the spatial dependencies captured by the GCN model, this approach can effectively model the complex traffic patterns within the urban transportation network. The approach for utilizing temporal dependency in prediction is explained in the next section.

2.3 Temporal Graph Convolutional Network

[6] presented a novel approach, the Temporal Graph Convolutional Network (T-GCN), for accurate and real-time traffic forecasting. As noted before, traffic prediction is crucial for urban traffic planning and management, but it poses challenges due to spatial and temporal dependencies in traffic data. The T-GCN model combines the Graph Convolutional Network (GCN) to capture spatial dependence and the Gated Recurrent Unit (GRU) to capture temporal dependence. The T-GCN model can be formulated as follows:

where $\mathbf{Z}_t = f(\mathbf{A}, \mathbf{X}_t)$ represents the spatial features extracted by a two layer graph convolution operation (2.3) at time t and $\mathbf{X}_t$ is the input at time t . In the GRU component, $\mathbf{u}_t$ is the update gate, $\mathbf{r}_t$ is the reset gate, $\mathbf{S}_t$ is the hidden state, and $\odot$ denotes the Hadamard product. $\mathbf{W}_u$, $\mathbf{W}_r$, and $\mathbf{W}_c$ are learnable weight matrices, and $\mathbf{b}_u$, $\mathbf{b}_r$, and $\mathbf{b}_c$ are the corresponding biases. The model is trained by minimizing the difference between the predicted traffic speeds $\hat{Y}_t$ of T-GCN model and actual speeds Y_t using the following loss function:

$$loss = \| Y_t - \hat{Y}_t \| + \lambda L_{reg} \quad (2.4)$$

where L_{reg} is an L2 regularization term to prevent overfitting and λ is a hyperparameter controlling its strength.

By using these components, the T-GCN model can effectively predict traffic patterns based on urban road networks. Specifically, the GCN is used to learn complex topological structures to capture spatial dependence, while the gated recurrent unit learns dynamic changes of traffic data to capture temporal dependence. The model has been successfully employed for traffic forecasting based on urban road networks, with experimental results on real-world traffic datasets demonstrating that the T-GCN model can effectively obtain spatio-temporal correlation from traffic data and outperform existing baseline methods.

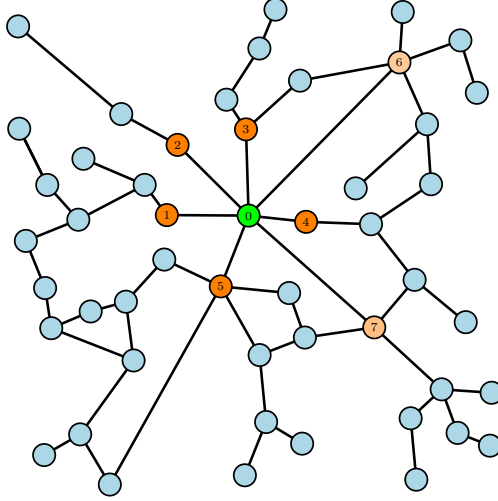


Fig. 2: A conceptual 2D representation of a traffic network. The influence of a node on the target (green) depends not only on spatial proximity (lighter color = farther) but also on the topology and travel time.

3 The Proposed Method: Estimating the Adjacency Matrix with Graph Structure Learning

In both GCN [11] and T-GCN [6] methods, the adjacency matrix of the graph ($\mathbf{A}$ in GCN and T-GCN), describing the spatial relationships of the nodes, is predefined based on urban planning information which rely on predefined spatial relationships between roads. Here, we propose to estimate the adjacency matrix $\mathbf{A}$ by learning from data, which is known as graph structure learning. Graph structure learning is a fundamental task in various fields, including machine learning, data mining, and network science [35]. In graph structure learning, the goal is to learn a graph representation that captures the underlying structure of the observed data. First, we describe the intuition behind our idea, and then explain some approaches used in GSL.

The goal of the proposed method is to infer the underlying structure of a graph or network from observed traffic data. The key idea is that in a typical urban traffic network, the spatial proximity of roads does not always accurately reflect the traffic dependencies between them. Nodes that are physically closer may not have as strong an influence on each other’s traffic patterns as nodes that are farther apart. The time it takes for traffic conditions to propagate from one node to another is an important factor that should be considered. Nodes that are farther away may influence the target node’s traffic, but with a greater time delay compared to closer nodes. By estimating the graph structure directly from traffic data, rather than relying solely on spatial proximity, the model can capture the actual temporal dependencies between roads. This can lead to more accurate traffic forecasting, as the model will be able to weigh the contributions of each neighboring node based on the real-world traffic patterns, rather than using the fixed spatial graph of the urban map.

Figure 2 illustrates 2D version of urban traffic network represented in figure 1, with node 0 (green) as the target node and orange ones representing its neighboring nodes. The visual distances between the nodes approximately correspond to the real-world distances between the roads. The key points shown in the figure are:

1. Nodes that are physically closer to node 0 (e.g., orange nodes 1-5) may have a stronger and more immediate influence on node 0’s traffic patterns compared to nodes that are farther away (e.g., light orange nodes 6-7).
2. The travel time between the nodes is an important factor, as the traffic conditions of the farther nodes (e.g., node 6) will influence node 0 with a greater time delay compared to the closer nodes (e.g., node 1), even though node 6 is adjacent to node 0.
3. The topology of the network, as represented by the graph, also plays a role in determining the influence of each node on the target node 0. For example, node 2 is adjacent to node 0 but may have a negligible influence on node 0’s traffic, as it appears to be a relatively weak connector linking only the two smaller nodes to the northwest. In contrast, the other neighboring nodes of node 0 (e.g., nodes 1, 3-5) seem to be more prominent, potentially acting as “hubs” with stronger traffic flows to node 0.

By learning the graph structure directly from traffic data, the model can capture these nuanced spatial and temporal topological dependencies, leading to more accurate traffic forecasting compared to approaches that rely solely on the spatial proximity of nodes.

3.1 Integration of Static Graph Structure with Temporal Modeling

Our approach combines static graph structure learning with dynamic temporal modeling:

- The GSL component learns a static graph structure that captures persistent traffic dependencies between roads
- This learned structure feeds into the GRU component, which models how these dependencies evolve over time
- The combination allows us to capture both stable spatial relationships and dynamic temporal patterns

While the learned graph structure is static, the GRU’s temporal modeling allows the influence of each road to vary dynamically based on current traffic conditions. This addresses the need to model both consistent spatial dependencies and varying temporal patterns.

3.2 GSL approaches

The goal of GSL is to discover the structure of the graph, which can be used to understand the inherent connections and dependencies within a system. In the context of traffic prediction, this means learning how different roads influence each other’s traffic patterns over time. One prominent approach for GSL is *score-based learning*, where we evaluate the likelihood of candidate structures based on empirical data. Another avenue is *constraint-based learning*, which enforces specific rules (such as acyclicity) to guide the search for optimal structures.

Directed acyclic graphs (DAGs) play a crucial role in this context. DAGs are graphs where edges have a direction, and no cycles exist. They effectively represent causal relationships and dependencies between variables. In traffic prediction, DAGs can model how traffic conditions propagate through the network over time, where the direction of edges represents the temporal flow of traffic influence. Learning DAGs from data is challenging due to the superexponential search space as the number of nodes increases. Traditional methods rely on local heuristics, but recent breakthroughs propose continuous optimization approaches that avoid the combinatorial constraints entirely. These methods formulate the problem as a continuous program over real matrices, utilizing numerical algorithms for efficient solutions. DAGs find applications in biology, genetics, machine learning, and causal inference [36, 37].

While [37] introduced a general framework for DAG learning from data, *our key contribution is applying this approach specifically to traffic prediction*. In their formulation, which uses slightly different notation from previous sections¹, $\mathbf{X} \in \mathbb{R}^{n \times d}$ is a data matrix consisting of n i.i.d. observations of the random vector $\mathbf{X} = (X_1, \dots, X_d)$ and $\mathbb{D}$ denote the (discrete) space of DAGs $\mathbf{G} = (\mathbf{V}, \mathbf{E})$ on d nodes. In our novel application to traffic prediction, we specifically interpret $\mathbf{X}$ as the traffic speed data matrix, where each row corresponds to a time step and each column represents a road. This innovative use of traffic data as the input matrix for graph structure learning enables us to discover underlying dependencies between roads that may not be apparent from spatial relationships alone.

Building on this idea of continuous optimization for DAG learning, [37] modeled X via a structural equation model (SEM) defined by a weighted adjacency matrix $W \in \mathbb{R}^{d \times d}$ ($\mathbf{A}$ in the previous section). Thus, instead of operating on the discrete space $\mathbb{D}$, they operate on $\mathbb{R}^{d \times d}$, the continuous space of $d \times d$ real matrices. This continuous formulation is particularly suitable for traffic prediction as it allows us to capture the strength of influence between roads, not just their binary connectivity. Zheng et. al. proposed a new approach for score-based learning of DAGs by converting the traditional *combinatorial* optimization problem (left) into a *continuous* program (right):

$$\begin{aligned} \min_{W \in \mathbb{R}^{d \times d}} F(W) & \iff \min_{W \in \mathbb{R}^{d \times d}} F(W) \\ \text{subject to } \mathbf{G}(W) \in \text{DAGs} & \quad \text{subject to } h(W) = 0, \end{aligned} \quad (3.1)$$

¹Note that while we previously used $\mathbf{A}$ to denote the adjacency matrix, in this section we follow the notation of [37] and use W to maintain consistency with their formulation.

Algorithm 1 NOTEARS algorithm [37]

Require: Initial guess (W_0, α_0) , progress rate $c \in (0, 1)$, tolerance $\epsilon > 0$, threshold $\omega > 0$

- 1: **for** $t = 0, 1, 2, \dots$ **do**
- 2: Solve primal: $W_{t+1} \leftarrow \arg \min_W L^\rho(W, \alpha_t)$
- 3: Dual ascent: $\alpha_{t+1} \leftarrow \alpha_t + \rho h(W_{t+1})$
- 4: **if** $h(W_{t+1}) < \epsilon$ **then**
- 5: Set $\widetilde{W}_{\text{ECP}} = W_{t+1}$
- 6: **break**
- 7: **end if**
- 8: **end for**
- 9: Return the thresholded matrix $\widehat{W} = \widetilde{W}_{\text{ECP}} \circ \mathbf{1}(|\widetilde{W}_{\text{ECP}}| > \omega)$

where $\mathbf{G}(W)$ is the d -node graph induced by the weighted adjacency matrix W , $F : \mathbb{R}^{d \times d} \rightarrow \mathbb{R}$ is a score function. The key technical device $h : \mathbb{R}^{d \times d} \rightarrow \mathbb{R}$ is a smooth function over real matrices, whose level set at zero exactly characterizes acyclic graphs. Zheng et. al. proposed to replace the combinatorial acyclicity constraint with a single smooth equality constraint $h(W) = 0$. The goal was to find a function $h : \mathbb{R}^{d \times d} \rightarrow \mathbb{R}$ that satisfies the following desiderata:

- (a) $h(W) = 0$ if and only if W is acyclic (i.e. $\mathbf{G}(W) \in \mathbb{D}$);
- (b) The values of h quantify the “DAG-ness” of the graph;
- (c) h is smooth;
- (d) h and its derivatives are easy to compute.

The main theorem of the work by [37] is as follows:

Theorem 1 A matrix $W \in \mathbb{R}^{d \times d}$ is a DAG if and only if

$$h(W) = \text{tr}(e^{W \circ W}) - d = 0, \quad (3.2)$$

where $\circ$ is the Hadamard product and e^A is the matrix exponential of A . Moreover, $h(W)$ has a simple gradient

$$\nabla h(W) = (e^{W \circ W})^T \circ 2W, \quad (3.3)$$

and satisfies all of the desiderata (a)-(d).

A key conclusion from Theorem 1 is that h and its gradient only involve evaluating the matrix exponential, which is a well-studied function in numerical analysis and available in many scientific computing libraries. This makes the approach computationally efficient and practical for real-world traffic prediction applications. The proof of this theorem can be found in [37]. Based on this theorem, Algorithm 1 presents the NOTEARS (*Non-combinatorial Optimization via Trace Exponential and Augmented lagRangian for Structure learning*) method for learning DAGs from data.

The NOTEARS algorithm employs the augmented Lagrangian method to solve the equality-constrained program (ECP). The augmented Lagrangian function $L^\rho(W, \alpha)$ is defined as:

$$L^\rho(W, \alpha) = F(W) + \frac{\rho}{2}|h(W)|^2 + \alpha h(W) \quad (3.4)$$

where $F(W)$ is the score function measuring how well the weighted adjacency matrix W fits the data, $h(W)$ is the acyclicity constraint from Theorem 1, α is the Lagrange multiplier, and $\rho > 0$ is the penalty parameter. This method converts the constrained optimization problem into a sequence of unconstrained subproblems that are easier to solve. The algorithm iteratively updates the weight matrix W and the Lagrange multiplier α until convergence, producing the solution $\widetilde{W}_{\text{ECP}}$. For detailed theoretical analysis and implementation details of the optimization method, see [37].

Here, we employ this graph structure learning to estimate the adjacency matrix $\mathbf{A}$ of the urban graph, which effectively captures the temporal dependencies underlying the urban network. Our approach differs from previous work in several key ways:

1. Unlike traditional GCN and T-GCN methods that rely on predefined spatial relationships, our method learns the graph structure directly from traffic data, allowing it to discover non-obvious dependencies between roads.
2. The continuous optimization approach enables us to capture the strength of influence between roads, not just their binary connectivity, providing more nuanced traffic pattern modeling.
3. Our method can adapt to changing traffic patterns over time, as the graph structure is learned from the data rather than being fixed based on urban planning information.

Our experimental results demonstrate that the GSL approach outperforms the traditional method of using the spatial adjacency matrix of the urban map, highlighting the benefits of learning the graph structure from data.

4 Experimental Results

In this section, we provide a thorough evaluation of our proposed method in comparison to baseline approaches using two real-world traffic datasets. We begin by describing the datasets and experimental setup, followed by a detailed presentation of the results and analysis². A preliminary version of this idea was presented in [38]³.

4.1 Datasets and Experimental Setup

Our evaluation utilizes two widely-used traffic datasets: the SZ-Taxi dataset and the Los-loop dataset. The SZ-Taxi dataset comprises taxi trajectory information collected in Shenzhen, China, in January 2015, with speed measurements from 156 sensors on

²The implementation of the paper using PyTorch, along with the data, is available at <https://github.com/mamintoosi/TGCN-GSL-PyTorch>

³The codes implemented in TensorFlow 1 are accessible at <https://github.com/mamintoosi/TGCN-GSL-TF>. However, since TensorFlow 1 is deprecated, the code for this paper has been developed using PyTorch 2

major roads in the Luohu District, aggregated at 15-minute intervals. The Los-loop dataset, on the other hand, consists of real-time traffic speed data from 207 loop detectors on Los Angeles County highways, collected from March 1 to March 7, 2012, and aggregated every 5 minutes. These datasets provide complementary perspectives, with the former representing urban traffic patterns and the latter capturing highway dynamics.

Each dataset is structured into two matrix formats: an adjacency matrix encoding spatial relationships between roads and a feature matrix describing speed variations over time for each road. The adjacency matrix captures connectivity strengths between roads, while the feature matrix organizes traffic speeds by road and time interval. To ensure robust evaluation, we split the data into training (80%) and testing (20%) sets, preserving the temporal order to reflect real-world forecasting scenarios. Performance is assessed across multiple time horizons (1 to 4 steps), corresponding to 5–20 minutes for Los-loop and 15–60 minutes for SZ-taxi.

The study by [6] demonstrated that the T-GCN model achieved superior performance in traffic speed prediction at time steps 1 and 2, while also outperforming most competing methods at time steps 3 and 4, as evaluated by RMSE metrics. Several baseline methods were compared, including the History Average model [39] that relies on historical traffic averages for predictions, the Autoregressive Integrated Moving Average (ARIMA) model [40] for time series forecasting, and the Support Vector Regression (SVR) model [14] with linear kernel and a penalty term of 0.001. Additionally, the comparison included the Graph Convolutional Network (GCN) [11] and Gated Recurrent Unit (GRU) model [41].

Since our proposed methodology builds upon both GCN and T-GCN architectures, and considering T-GCN’s established advantages over other state-of-the-art approaches, we focused our comparative analysis specifically on these two foundational models. This selective comparison is justified by the fact that GCN and T-GCN serve as the basis for numerous contemporary methods, including those presented in [42] and [5]. Our primary objective is to demonstrate the potential benefits of employing graph structure learning (GSL) for adjacency matrix estimation within these established frameworks, rather than conducting an exhaustive comparison with all existing approaches.

All experiments were conducted on a workstation equipped with an NVIDIA GeForce RTX 3090 GPU, an Intel Core i3 CPU, and 16 GB of RAM. The models were implemented using PyTorch framework on a windows environment. To ensure reproducibility and fair comparison, we fixed the random seeds for all libraries (PyTorch, NumPy) to a constant value across all experiments. While the neural network training involves stochastic elements (e.g., weight initialization), utilizing a fixed seed ensures that the reported improvements are consistent and replicable. Furthermore, the DAGMA algorithm used for structure learning is deterministic, yielding a stable adjacency matrix for a given dataset.

The specific hyperparameters used for training the GCN and T-GCN models are summarized in Table 1. We utilized the Adam optimizer with a learning rate of 0.001 and a batch size of 64. The historical window size was set to 12 time steps.

Table 1: Hardware specifications and hyperparameter settings used in the experiments.

Parameter/Component	Value/Description
<i>Hardware & Environment</i>	
GPU	NVIDIA GeForce RTX 3090 (24GB)
CPU	Intel Core i3 (9th Gen)
System RAM	16 GB
Framework	PyTorch
<i>Model Hyperparameters</i>	
Historical Window (T)	12
Prediction Horizons	1, 2, 3, 4
Batch Size	64
Learning Rate	0.001
Max Epochs	50
Optimizer	Adam
Loss Function	Mean Squared Error (MSE)
Structure Learning Algo.	DAGMA

4.2 Model Configurations

Our study employs two base architectures: the Graph Convolutional Network (GCN) and the Temporal Graph Convolutional Network (TGCN). The GCN captures spatial dependencies using predefined road network structures, while the TGCN integrates GCN with Gated Recurrent Units (GRUs) to model both spatial and temporal dependencies. For each architecture, we evaluate three configurations: the original model with a predefined adjacency matrix, our proposed method using a learned acyclic graph structure (*GCN-GSL* and *TGCN-GSL*), and a cyclic variant (*GCN-cGSL* and *TGCN-cGSL*) created by symmetrizing the learned adjacency matrix.

The graph structure learning process leverages the DAGMA method [10], which builds on prior work[37] to efficiently estimate adjacency matrices. This process is computationally efficient, taking approximately 2 minutes on standard hardware. Hyperparameters, including learning rate, batch size, and sequence length, were tuned through extensive experimentation to ensure optimal performance. Notably, our evaluation focuses on comparing our proposed methods against GCN and TGCN baselines, as these represent the foundational approaches in the literature and provide a clear benchmark for assessing the benefits of graph structure learning.

The performance of our GSL-based traffic prediction model is influenced by several carefully selected hyperparameters, each playing a crucial role in the model’s effectiveness. The learning rate, set at 0.001, was chosen to ensure stable convergence while maintaining training efficiency. This value strikes a balance between rapid learning and avoiding oscillations during optimization. For the graph sparsity parameter λ , we observed that different values were optimal for different datasets: 0.01 for the SZ-taxi dataset and 0.02 for the Los-loop dataset.

The historical window size, fixed at 12 time steps, was determined empirically to provide sufficient context for accurate predictions without overwhelming the model

with redundant information. Sensitivity analysis revealed that the model is particularly sensitive to the graph sparsity parameter λ , as it directly controls the density of connections in the learned graph structure. In contrast, the learning rate and historical window size showed moderate effects within reasonable ranges, allowing for some flexibility in their selection without significantly compromising performance.

4.3 Evaluation Metrics

To comprehensively assess model performance, we employ four standard metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Accuracy, and R-squared (R2). RMSE provides a measure of prediction error magnitude, while MAE offers an intuitive interpretation of average error. Accuracy, calculated as the relative deviation from actual values, and R2, indicating the proportion of variance explained by the model, further validate the predictive power of our approach. These metrics collectively ensure a thorough evaluation of model performance across different aspects of traffic forecasting. These metrics are calculated as follows:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_t - \hat{Y}_t)^2} \quad (4.1)$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |Y_t - \hat{Y}_t| \quad (4.2)$$

$$Accuracy = \frac{1}{n} \sum_{i=1}^n \left(1 - \frac{|Y_t - \hat{Y}_t|}{Y_t} \right) \quad (4.3)$$

$$R2 = 1 - \frac{\sum_{i=1}^n (Y_t - \hat{Y}_t)^2}{\sum_{i=1}^n (Y_t - \bar{Y}_t)^2} \quad (4.4)$$

4.4 Ablation Studies

To understand the contribution of each component in our proposed method, we conducted comprehensive ablation studies:

- **Impact of Graph Structure:** We compared performance using physical proximity graphs vs. learned graphs, demonstrating improvement in prediction accuracy with learned structures.
- **GSL vs. cGSL:** The cyclic variant (cGSL) showed consistent improvements over acyclic GSL in GCN, while GSL performed better with T-GCN, suggesting different optimal graph structures for spatial and temporal modeling.

The results in Table 2 show that the proposed GCN-cGSL method consistently outperforms both the standard GCN and GCN-GSL methods across all prediction lengths for both the SZ-taxi and Los-loop datasets. This demonstrates the effectiveness of using the graph structure learning approach to estimate the adjacency matrix from

Table 2: Performance comparison of GCN and GCN with the proposed GSL methods on the SZ-taxi and Los-loop datasets for different prediction horizons (PH). The results are evaluated using RMSE, MAE, Accuracy, and R2 metrics. The proposed GCN-cGSL method outperforms other methods across all metrics.

PH	Method	SZ-taxi				Los-loop			
		RMSE	MAE	Accuracy	R2	RMSE	MAE	Accuracy	R2
1	GCN	5.958	4.409	0.585	0.675	7.724	5.555	0.869	0.689
	GCN-GSL	4.886	3.161	0.660	0.782	7.527	5.065	0.872	0.705
	GCN-cGSL	4.648	3.078	0.676	0.802	5.440	3.452	0.907	0.846
2	GCN	5.983	4.437	0.583	0.672	7.940	5.692	0.865	0.672
	GCN-GSL	4.904	3.173	0.658	0.780	7.867	5.268	0.866	0.678
	GCN-cGSL	4.672	3.051	0.674	0.800	5.806	3.613	0.901	0.825
3	GCN	5.991	4.438	0.582	0.671	8.102	5.782	0.862	0.659
	GCN-GSL	4.958	3.223	0.655	0.775	8.073	5.453	0.863	0.662
	GCN-cGSL	4.712	3.122	0.672	0.797	6.171	3.821	0.895	0.802
4	GCN	6.002	4.450	0.582	0.670	8.285	5.876	0.859	0.644
	GCN-GSL	4.933	3.199	0.656	0.777	9.067	6.085	0.846	0.575
	GCN-cGSL	4.726	3.107	0.671	0.795	6.745	4.097	0.885	0.764
% Improvement		21.6	30.3	15.5	18.8	24.7	34.6	3.9	21.5

data, leading to improved traffic forecasting performance and highlighting the benefits of incorporating cyclic dependencies in the graph structure.

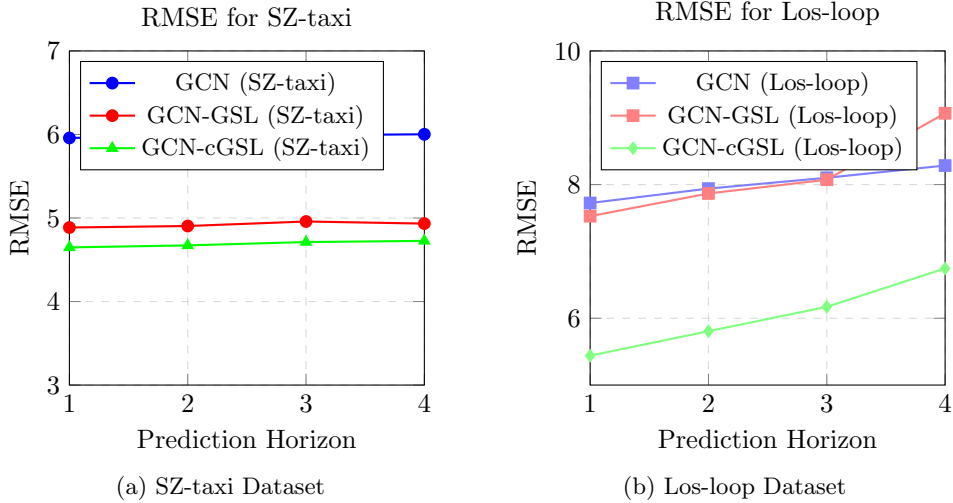


Fig. 3: RMSE comparison between GCN and GCN with the proposed GSL methods across all prediction horizons (1-4) for both datasets. As shown, the proposed GCN-cGSL method outperforms the standard GCN method across all horizons.

Table 3: Performance comparison of TGCN and TGCN with the proposed GSL methods on the SZ-taxi and Los-loop datasets for different prediction horizons (PH). The results are evaluated using RMSE, MAE, Accuracy, and R2 metrics. The proposed TGCN-GSL method outperforms other methods across all metrics.

PH	Method	SZ-taxi				Los-loop			
		RMSE	MAE	Accuracy	R2	RMSE	MAE	Accuracy	R2
1	TGCN	4.866	3.606	0.661	0.783	6.588	4.573	0.888	0.774
	TGCN-GSL	4.214	2.797	0.706	0.837	4.818	2.980	0.918	0.879
---	TGCN-cGSL	4.821	3.537	0.664	0.787	6.550	4.553	0.889	0.776
	TGCN	4.506	3.213	0.686	0.814	6.960	4.838	0.882	0.748
2	TGCN-GSL	4.239	2.801	0.705	0.835	5.400	3.410	0.908	0.848
	TGCN-cGSL	4.534	3.276	0.684	0.812	6.915	4.830	0.882	0.751
---	TGCN	4.685	3.416	0.674	0.799	7.361	5.053	0.875	0.718
	TGCN-GSL	4.344	3.062	0.697	0.828	5.846	3.467	0.900	0.822
3	TGCN-cGSL	4.630	3.334	0.677	0.804	7.331	5.054	0.875	0.721
	TGCN	4.934	3.638	0.656	0.777	7.568	5.166	0.871	0.703
---	TGCN-GSL	4.366	2.885	0.696	0.826	6.257	3.805	0.893	0.797
	TGCN-cGSL	4.774	3.487	0.667	0.791	7.539	5.172	0.872	0.705
% Improvement		9.5	16.6	4.8	4.9	21.8	30.5	3.0	13.7

Figure 3 provides a visual comparison of the RMSE values for both methods across all prediction horizons. For the SZ-taxi dataset, the GCN-cGSL method achieves the lowest RMSE values (around 4.6-4.7) compared to the standard GCN method (around 5.9-6.0), demonstrating a significant improvement in prediction accuracy. Similarly, for the Los-loop dataset, the GCN-cGSL method outperforms the standard GCN method across all horizons, with RMSE values ranging from 5.4 to 6.7, while the standard GCN method shows RMSE values between 7.7 and 8.3.

The proposed method TGCN-GSL outperforms the standard TGCN method and other versions of the proposed method (TGCN-cGSL) across all horizons for both datasets, as shown in Table 3. This highlights the importance of capturing temporal dependencies in urban traffic networks and the benefits of learning the graph structure from data. According to the results in Tables 2 and 3, the proposed methods, *GCN-cGSL* and *TGCN-GSL*, consistently outperform the standard methods. The results demonstrate the benefits of learning the graph structure from data, leading to more accurate traffic forecasting.

Figure 4 provides a similar visual comparison for the TGCN methods. For both datasets, the TGCN-GSL method achieves lower RMSE values compared to the standard TGCN method. On the SZ-taxi dataset, the TGCN-GSL method shows an improvement of approximately 9.5% in RMSE (average over all horizons). The Los-loop dataset shows even more dramatic improvements, with TGCN-GSL achieving an RMSE reduction of about 21.8%. These results further demonstrate the effectiveness of learning the graph structure from data.

Figures 5, 6, 7, and 8 illustrate the training progress of both the standard and proposed methods over 50 epochs for both datasets. These figures show the evolution of

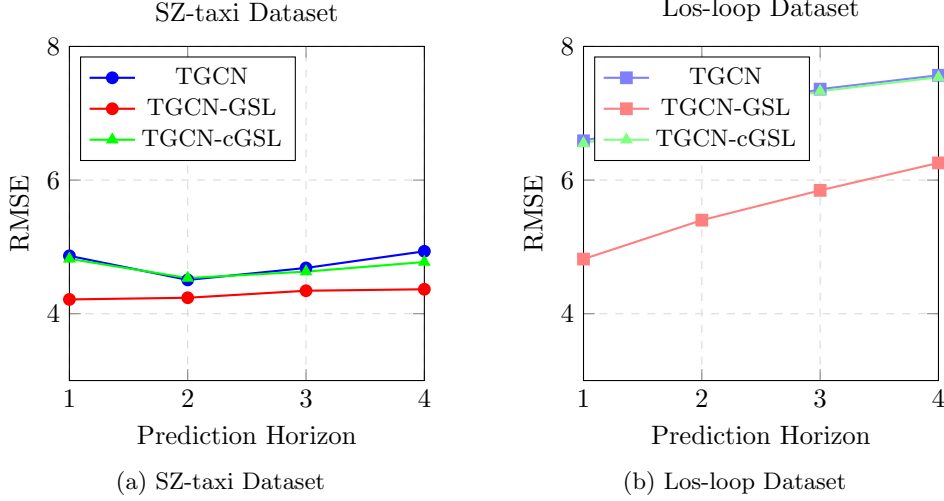


Fig. 4: RMSE comparison between TGCN, TGCN-GSL, and TGCN-cGSL methods across all prediction horizons (1-4) for both datasets. As shown, the proposed TGCN-GSL method outperforms the standard GCN method across all horizons.

key performance metrics (RMSE, MAE, Accuracy, and R2) during training, evaluated on validation data, providing insights into how well each method learns and converges over time. As shown in Figures 5 and 6, the performance of the proposed *GCN-cGSL* (green color) method on both datasets is superior to the standard GCN method (blue color) throughout the training process.

Similarly, Figures 7 and 8 illustrate that the proposed method, *TGCN-GSL* (orange color), achieves better performance than the standard T-GCN method (blue color) on both datasets throughout the training process, with more stable convergence and lower error metrics. In our experiments, we observed that the proposed GCN-cGSL method outperformed the standard GCN method, while the TGCN-GSL method outperformed the standard TGCN method. This indicates that for the GCN base method, converting the learned adjacency matrix to an adjacency matrix of a cyclic graph was superior. In contrast, for the TGCN base method, the output of DAGMA, which is the adjacency matrix of an acyclic directed graph, was the winner. This difference in performance can be attributed to the inherent characteristics of the base methods. GCN primarily focuses on capturing spatial dependencies, and the introduction of cyclic connections in the graph structure allows it to better model complex spatial relationships and feedback loops within the urban traffic network. On the other hand, TGCN combines both spatial and temporal modeling through GCN and GRU components. The acyclic graph structure learned by DAGMA effectively captures the temporal dependencies and directional flow of traffic, which aligns well with the temporal modeling capabilities of TGCN. Therefore, the acyclic graph structure provides a more accurate representation of the underlying traffic dynamics for TGCN, leading

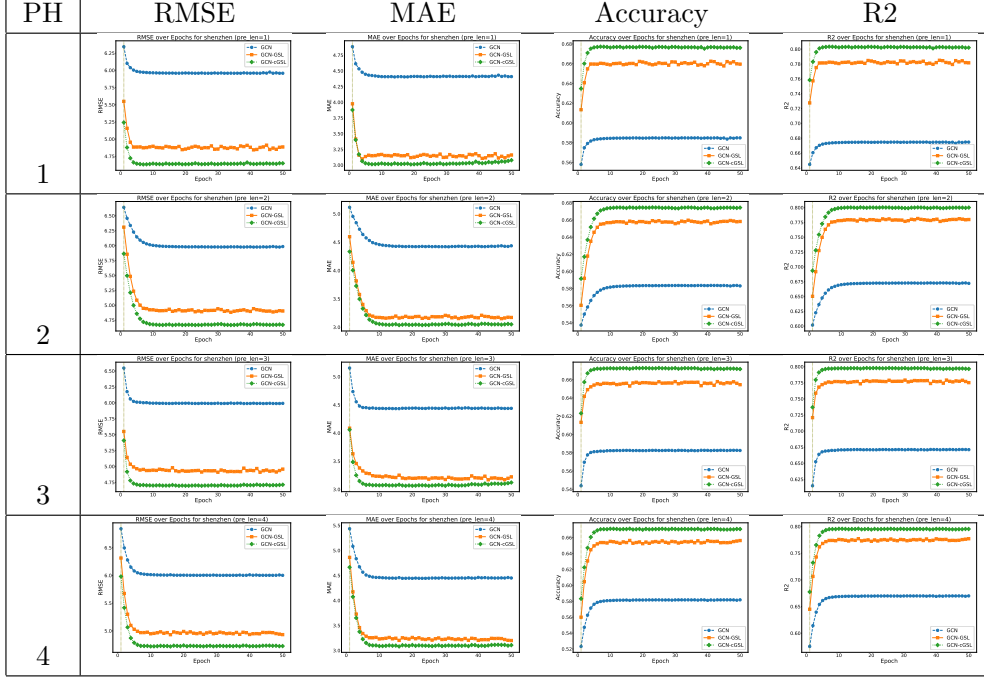


Fig. 5: Performance comparison of GCN on the SZ-taxi dataset over 50 training epochs. The subfigures show RMSE, MAE, Accuracy, and R2 metrics, demonstrating the superior convergence of the proposed *GCN-cGSL* method (green color) compared to the standard GCN method (blue color).

to superior performance. In 5, we discuss about the acyclicity of the learned graph and its implications for traffic prediction.

5 Analysis of Learned Graph Structures: Reconciling DAG Assumptions with Cyclic Traffic Networks

Our methodological approach employs the DAGMA algorithm [10] for graph structure learning, which is designed to estimate a Directed Acyclic Graph (DAG). This presents an apparent paradox, as urban traffic networks are inherently cyclic when considered as static spatial graphs. However, as our experimental results demonstrate, the estimated DAG yields good performance, particularly when integrated with temporal models. This section provides a detailed analysis of this phenomenon by distinguishing between two distinct graph perspectives.

5.1 Spatial vs. Temporal Dependency Graphs

The resolution to this paradox lies in recognizing that our GSL framework learns a fundamentally different type of graph than the physical road network:

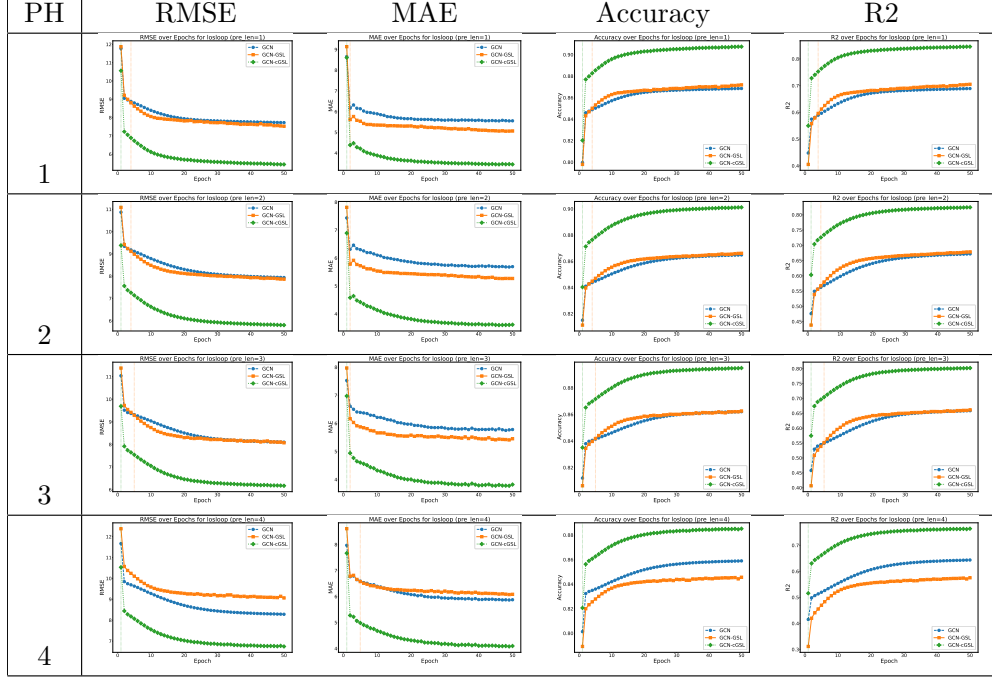


Fig. 6: Performance comparison of GCN on the Los-loop dataset over 50 training epochs. The subfigures show RMSE, MAE, Accuracy, and R2 metrics, demonstrating the superior convergence of the proposed *GCN-cGSL* method (green color) compared to the standard GCN method (blue color).

- **Spatial Graph ($\mathcal{G}_{spatial}$):** This is the conventional representation where nodes are road segments and edges represent physical connectivity (e.g., intersections, adjacent road segments). This graph is inherently *undirected and cyclic*, as traffic can flow in both directions and loops exist in the road network. The standard GCN and T-GCN baselines use a predefined version of this graph, $\mathbf{A}_{spatial}$.
- **Temporal Dependency Graph ($\mathcal{G}_{temporal}$):** This is the graph learned by our GSL method. It represents *cross-temporal dependencies* between the states of road segments. An edge $j \rightarrow i$ in this graph signifies that the traffic state on road j at time t has a predictive, causal influence on the traffic state of road i at time $t+1$. This graph is naturally *directed and acyclic* because influence flows forward in time; there are no cycles across time steps.

The GSL algorithm thus discovers the weighted adjacency matrix $\mathbf{W}_{temporal}$ that best explains the temporal dynamics observed in the data (See Figure 9).

5.2 Why DAGs Work for Temporal Models (T-GCN)

The T-GCN architecture combines a GCN (for spatial features) with a GRU (for temporal dynamics). When we integrate the learned $\mathbf{W}_{temporal}$ into T-GCN, we are

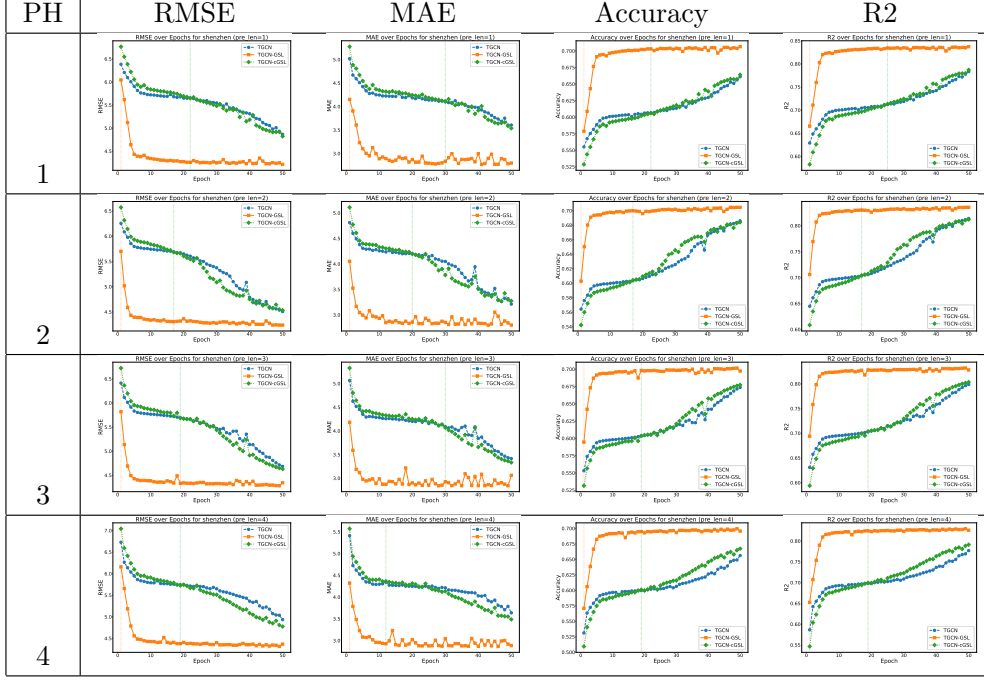


Fig. 7: Performance comparison of TGCN on the SZ-taxi dataset over 50 training epochs. The subfigures show RMSE, MAE, Accuracy, and R2 metrics, demonstrating the superior convergence of the proposed *TGCN-GSL* method (orange color) compared to the standard TGCN method (blue color).

providing the model with a structured prior on how information flows through time. The acyclic nature of $\mathbf{W}_{temporal}$ is a perfect match for the GRU’s sequential processing: it tells the model which past states from other nodes are most relevant for predicting the current state. This leads to the superior performance of TGCN-GSL over TGCN-cGSL, as the symmetrized cyclic version introduces spurious feedback loops that confuse the temporal model.

5.3 Why Cyclic Graphs Work for Spatial Models (GCN)

Conversely, the GCN operates on a single static graph. When we supply it with the learned $\mathbf{W}_{temporal}$ (a DAG) and treat it as a static spatial graph, we are forcing a temporal structure into a spatial layer. This is why symmetrizing it ($\mathbf{W}_{cyclic} = \mathbf{W}_{temporal} + \mathbf{W}_{temporal}^T$) to create GCN-cGSL yields better performance. The cyclic version better approximates the bidirectional nature of real-world spatial interactions, even though the underlying learned dependencies were originally temporal. The GCN benefits from the richer, data-driven connections discovered by GSL, but requires them to be in a cyclic format to function optimally.

This analysis reveals that the choice of graph structure should align with the inductive biases of the prediction model:

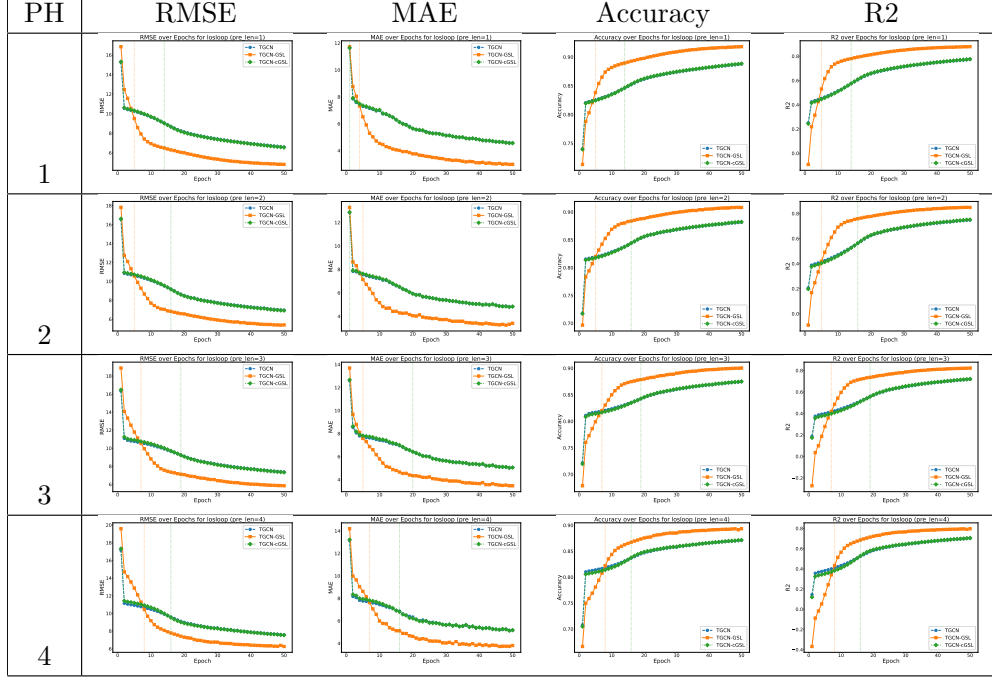


Fig. 8: Performance comparison of TGCN on the Los-loop dataset over 50 training epochs. The subfigures show RMSE, MAE, Accuracy, and R2 metrics, demonstrating the superior convergence of the proposed *TGCN-GSL* method (orange color) compared to the standard TGCN method (blue color).

- For models with explicit temporal components (e.g., T-GCN, GRUs), the directed acyclic $\mathbf{W}_{temporal}$ provides an optimal prior on temporal information flow.
- For purely spatial models (e.g., GCN), the learned dependencies are best utilized in a symmetrized, cyclic form.

6 Conclusion and Future Works

Our work demonstrates how graph knowledge discovery can transform traditional approaches to traffic forecasting by revealing hidden network structures from data. The experimental results validate that this knowledge-driven approach significantly outperforms conventional methods, with key findings including:

1. Adjacent nodes in the urban spatial graph do not necessarily exhibit strong mutual traffic influences. The physical proximity between roads in an urban network may not accurately reflect their actual causal interdependencies, as other factors beyond spatial adjacency can determine how traffic patterns causally affect neighboring roads.

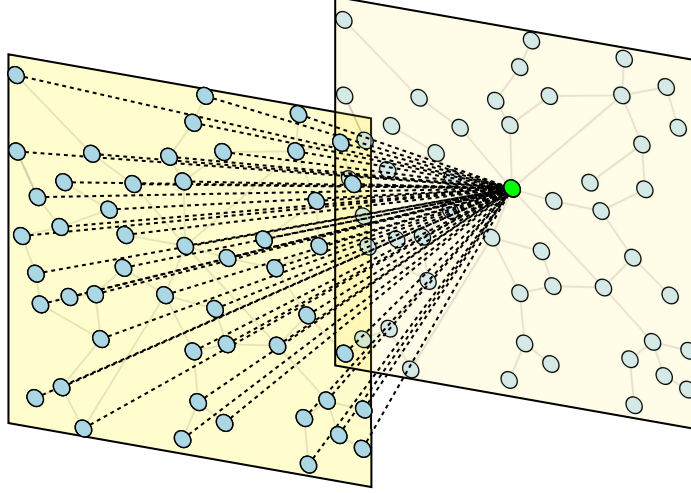


Fig. 9: Interpreting traffic dynamics as a temporal DAG. Each layer represents the state of all road segments at consecutive time steps. The GSL algorithm learns the cross-temporal dependencies (dashed arrows) that best predict the next state, resulting in a DAG that captures the directional flow of traffic influence. The solid gray edges represent the underlying spatial graph.

2. Accounting for the temporal propagation of traffic conditions is crucial, as nodes that are farther away can still causally influence the target node’s traffic, but with a greater time delay compared to closer nodes.
3. The direct causal structure learning from traffic data enhances the model’s ability to capture complex temporal and topological dependencies. This approach allows for a more precise identification of causal relationships between nodes, leading to superior forecasting accuracy.
4. The proposed method outperforms traditional proximity-based graph models by learning causal dependencies from data, thereby avoiding oversimplified assumptions about network structure and yielding superior prediction results. The discovered causal graphs offer interpretable insights into traffic flow dynamics.
5. The causal graph learning framework provides a simpler yet effective alternative to attention-based approaches while enhancing the model’s ability to capture complex dependencies. Unlike attention mechanisms that learn implicit relationships, our method explicitly reveals causal pathways in traffic networks.

Future research directions in this framework include: (1) optimizing the causal graph estimation process for larger networks, (2) developing hybrid methods that combine structure learning with attention mechanisms [8], and (3) investigating the integration of Graph Structure Learning (GSL) with emerging techniques like Graph Structure Prompt Learning [43] is promising. While GSL focuses on discovery of the core causal structure, prompt learning could potentially be utilized for dynamic refinement and fine-tuning of the learned graph during the neural network training process.

Furthermore, extending our framework to model time-varying causal relationships through incremental graph updates would better reflect real-world traffic dynamics.

Overall, these findings underscore how graph knowledge discovery enables more accurate and interpretable traffic forecasting. By moving beyond physical topology to discover actual traffic relationships, our method provides urban planners with valuable insights for evidence-based transportation management while significantly improving prediction performance.

Acknowledgements. The author would like to thank Amin Amiri and Morteza Mostafazadeh for their valuable collaboration in converting the T-GCN codebase from PyTorch Lightning to pure PyTorch implementation.

Appendix A Bibliometric Analysis Methodology

To justify our focus on graph-based approaches in traffic forecasting and trace the evolution of the field, we conducted a systematic bibliometric analysis.

A.1 Data Collection and Initial Filtering

A broad search was conducted in Scopus using the keyword “traffic forecast*”, which retrieved approximately 60,000 documents. This dataset was refined by filtering for peer-reviewed articles written in English within Engineering or Computer Science, resulting in a core collection of 784 articles published between 2000 and 2025.

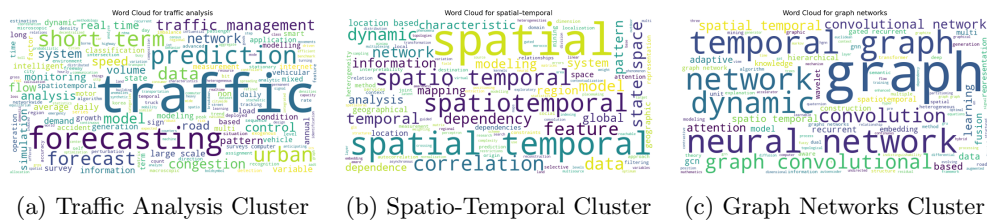
A.2 Keyword Processing and Semantic Clustering

The raw data contained 6,343 unique keywords. After removing terms appearing fewer than 10 times, the list was reduced to 217 keywords. An initial analysis using VOSviewer (Figure A1) revealed that similar terms—such as “graph neural network”, “GNN”, and “graph convolution network”—appeared as separate nodes, which hindered pattern recognition at a broader level.

To address this fragmentation, we developed an advanced keyword cleaning pipeline (code available in the project repository) with the following steps:

1. **Preprocessing:** Removing special characters, standardizing case, and eliminating common words.
2. **Abbreviation Expansion:** Replacing 30 common abbreviations with their full forms (e.g., converting “GNN” to “graph neural network”).
3. **Semantic Clustering:** Using Sentence-BERT embeddings to automatically group words based on semantic similarity.

This process condensed the 217 fragmented terms into 66 meaningful concept groups. Each cluster was labeled with a central representative term, and thesaurus mappings were generated to facilitate visualization. Figure A2 displays word clouds for three representative clusters.



directly from traffic data, combining the strengths of graph-based modeling with data-driven structure discovery. All related analysis code and procedures are available in the project’s GitHub repository.

- [1] Akhtar, M., Moridpour, S.: A review of traffic congestion prediction using artificial intelligence. *Journal of Advanced Transportation* **2021** (2021)
- [2] Ye, J., Zhao, J., Ye, K., Xu, C.: How to build a graph-based deep learning architecture in traffic domain: A survey. *IEEE Transactions on Intelligent Transportation Systems* (2020)
- [3] Xie, P., Li, T., Liu, J., Du, S., Yang, X., Zhang, J.: Urban flow prediction from spatiotemporal data using machine learning: A survey. *Information Fusion* **59**, 1–12 (2020)
- [4] Chen, F., Zhang, Y., Chen, L., Meng, X., Qi, Y., Wang, J.: Dynamic traveling time forecasting based on spatial-temporal graph convolutional networks. *Front. Comput. Sci.* **17**(6) (2023) <https://doi.org/10.1007/s11704-023-2704-x>
- [5] Zhou, Z., Basker, R., Yeung, D.-Y.: Graph neural networks for multivariate time-series forecasting via learning hierarchical spatiotemporal dependencies. *Engineering Applications of Artificial Intelligence* **147**, 110304 (2025) <https://doi.org/10.1016/j.engappai.2025.110304>
- [6] Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H.: T-GCN: A temporal graph convolutional network for traffic prediction. *IEEE Trans. on Intell. Transportation Systems* **21**(9), 3848–3858 (2020)
- [7] Bai, J., Zhu, J., Song, Y., Zhao, L., Hou, Z., Du, R., Li, H.: A3t-gcn: Attention temporal graph convolutional network for traffic forecasting. *ISPRS International Journal of Geo-Information* **10**(7) (2021) <https://doi.org/10.3390/ijgi10070485>
- [8] Sun, C., Li, C., Lin, X., Zheng, T., Meng, F., Rui, X., Wang, Z.: Attention-based

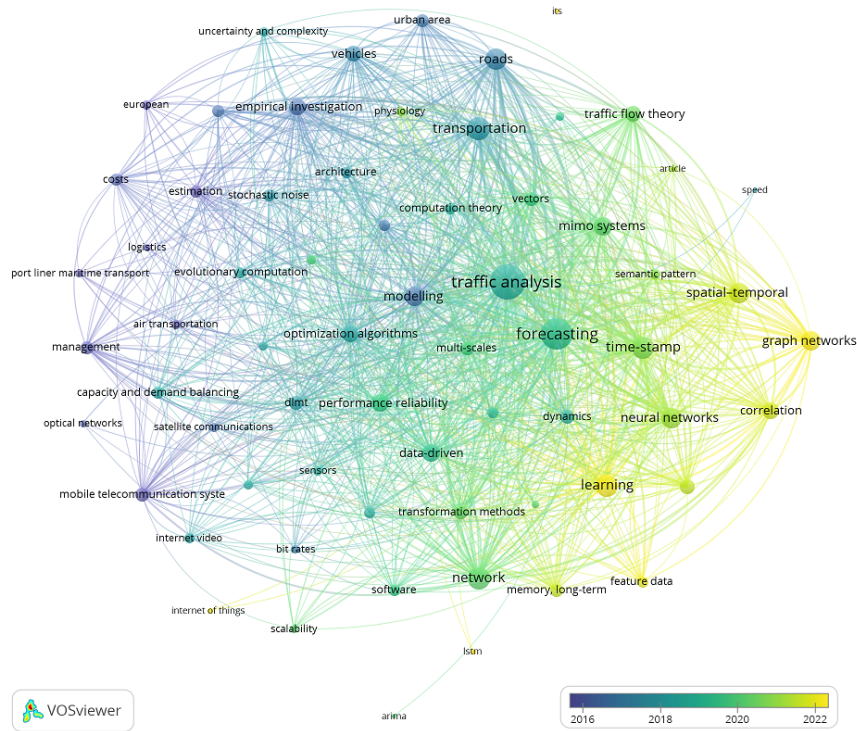


Fig. A3: Clustered keyword co-occurrence network from 784 traffic forecasting articles (2000-2025), generated using semantic similarity grouping. Nodes represent keyword clusters with minimum 10 occurrences, colored by average publication year. The visualization clearly shows graph-based methods (right nodes) as the most recent methodological focus.

graph neural networks: a survey. *Artificial Intelligence Review* **56**(2), 2263–2310 (2023) <https://doi.org/10.1007/s10462-023-10577-2>

- [9] Chen, G., Chen, L., Li, Z., Wei, Y., He, Z., Yang, F.: Ta-stgcn: trend-aware attention spatio-temporal graph convolutional network for traffic flow prediction. *International Journal of Data Science and Analytics* **22**(1), 56 (2026) <https://doi.org/10.1007/s41060-026-01040-w>
- [10] Bello, K., Aragam, B., Ravikumar, P.: Dagma: learning dags via m-matrices and a log-determinant acyclicity characterization. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22*. Curran Associates Inc., Red Hook, NY, USA (2024)
- [11] Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: *ICLR* (2017)

- [12] Hamed, M.M., Al-Masaeid, H.R., Said, Z.M.B.: Short-term prediction of traffic volume in urban arterials. *Journal of Transportation Engineering* **121**(3), 249–254 (1995)
- [13] Fabian, X., Ban, G., Boussaïd, R., *et al.*: Modeling and forecasting vehicular traffic flow as a seasonal arima process. *Journal of Transportation Engineering* **129**(6), 664–672 (2003)
- [14] Smola, A.J., Schölkopf, B.: A tutorial on support vector regression. *Statistics and Computing* **14**(3), 199–222 (2004)
- [15] Lippi, M., Bertini, M., Frasconi, P.: Short-term traffic flow forecasting. *IEEE Transactions on Intelligent Transportation Systems* **14**(2), 871–882 (2013)
- [16] Ojeda, L.L., Kibangou, A.Y., Wit, C.C.: Adaptive kalman filtering for multi-step ahead traffic flow prediction. In: *American Control Conference* (2013)
- [17] Park, D., Rilett, L.R.: Forecasting freeway link travel times with a multilayer feed-forward neural network. *Computer-Aided Civil and Infrastructure Engineering* **14**(5), 357–367 (2010)
- [18] Zhang, J., Zheng, Y., Qi, D.: Deep spatio-temporal residual networks for citywide crowd flows prediction, 1655–1661 (2017)
- [19] Cao, X., Zhong, Y., Zhou, Y., Wang, J., Zhu, C., Zhang, W.: Interactive temporal recurrent convolution network for traffic prediction. *IEEE Access* **5**, 9893–9901 (2017)
- [20] Fu, R., Zhang, Z., Li, L.: Using lstm and gru neural network methods for traffic flow prediction. In: *Youth Academic Conference of Chinese Association of Automation*, pp. 324–328 (2017)
- [21] Khan, H.M., Khan, A., Villar, S.G., Lopez, L.A.D., Almaleh, A., Al-Qahtani, A.M.: A comparative study of optimized-lstm models using tree-structured parzen estimator for traffic flow forecasting in intelligent transportation. *Computers, Materials and Continua* **83**(2), 3369–3388 (2025) <https://doi.org/10.32604/cmc.2025.060474>
- [22] Zuo, J., Zeitouni, K., Taher, Y., Garcia-Rodriguez, S.: Graph convolutional networks for traffic forecasting with missing values. *Data Mining and Knowledge Discovery* **37**(2), 913–947 (2023) <https://doi.org/10.1007/s10618-022-00903-7>
- [23] Wang, B., Long, Z., Sheng, J., Zhong, Q.: Spatial-temporal similarity fusion graph adversarial convolutional networks for traffic flow forecasting. *Journal of the Franklin Institute* **361**(17), 107299 (2024) <https://doi.org/10.1016/j.jfranklin.2024.107299>

- [24] Xia, Z., Zhang, Y., Yang, J., Xie, L.: Dynamic spatial-temporal graph convolutional recurrent networks for traffic flow forecasting. *Expert Systems with Applications* **240**, 122381 (2024) <https://doi.org/10.1016/j.eswa.2023.122381>
- [25] Remmouche, B., Boukraa, D., Zakharova, A., Bouwmans, T., Taffar, M.: Long-term spatio-temporal graph attention network for traffic forecasting. *Expert Systems with Applications* **288**, 128244 (2025) <https://doi.org/10.1016/j.eswa.2025.128244>
- [26] Zeb, A., Zheng, J., Ye, Y., Chen, J., Zhang, S., Wei, X., Jianqiao Yu, J.: Traffic forecasting with meta attentive graph convolutional recurrent network. *Expert Systems with Applications* **287**, 128073 (2025) <https://doi.org/10.1016/j.eswa.2025.128073>
- [27] Jia, C., Jiang, F., Chen, Z., Wang, R., Xiao, Y.: A dynamic traffic flow prediction method based on spatio-temporal graph transformer. *Applied Intelligence* **55**(16), 1048 (2025) <https://doi.org/10.1007/s10489-025-06895-3>
- [28] Liu, R., Shin, S.-Y.: A review of traffic flow prediction methods in intelligent transportation system construction. *Applied Sciences* **15**(7) (2025) <https://doi.org/10.3390/app15073866>
- [29] Peng, L., Liao, X., Li, T., Guo, X., Wang, X.: An overview based on the overall architecture of traffic forecasting. *Data Science and Engineering* **9**(3), 341–359 (2024) <https://doi.org/10.1007/s41019-024-00246-x>
- [30] Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., et al.: Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018)
- [31] Liu, H., Wei, J., Xu, T.: Community detection based on community perspective and graph convolutional network. *Expert Systems with Applications* **231** (2023) <https://doi.org/10.1016/j.eswa.2023.120748>
- [32] Liping Deng, B.G., Zheng, W.: GCN-based weakly-supervised community detection with updated structure centres selection. *Connection Science* **36**(1), 2291995 (2024) <https://doi.org/10.1080/09540091.2023.2291995>
- [33] Amintoosi, M.: Overlapping clusters in cluster graph convolutional networks. *Journal of Algorithms and Computation* **53**(2), 33–45 (2021)
- [34] Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W.L., Leskovec, J.: Graph convolutional neural networks for web-scale recommender systems. In: *KDD* (2018)

- [35] Chen, Y., Wu, L.: In: Wu, L., Cui, P., Pei, J., Zhao, L. (eds.) Graph Neural Networks: Graph Structure Learning, pp. 297–321. Springer, Singapore (2022). https://doi.org/10.1007/978-981-16-6054-2_14 . https://doi.org/10.1007/978-981-16-6054-2_14
- [36] Zuo, H., Jiang, J., Zhou, Y.: DAGOR: Learning DAGs via topological sorts and QR factorization. *Mathematics* **12**(8) (2024) <https://doi.org/10.3390/math12081198>
- [37] Zheng, X., Aragam, B., Ravikumar, P., Xing, E.P.: Dags with no tears: Continuous optimization for structure learning. In: NeurIPS (2018)
- [38] Amintoosi, M.: Urban traffic prediction with learning based graph convolutional networks. In: Proceedings of the 55th Annual Iranian Mathematics Conference, Ferdowsi University of Mashhad, pp. 145–148 (2024). (In Persian). <https://github.com/mamintoosi/TGCN-GSL-TF>
- [39] Liu, J., Guan, W.: A summary of traffic flow forecasting methods. *Journal of Highway Transportation Research Development* (2004)
- [40] Ahmed, M.S., Cook, A.R.: Analysis of Freeway Traffic Time-series Data by Using Box-Jenkins Techniques vol. 722, (1979)
- [41] Cho, K., Merrienboer, B.V., Bahdanau, D., Bengio, Y.: On the properties of neural machine translation: Encoder-decoder approaches. *Computer Science* (2014)
- [42] Yu, B., Lee, Y., Sohn, K.: Forecasting road traffic speeds by considering area-wide spatio-temporal dependencies based on a graph convolutional neural network (gcn). *Transportation Research Part C: Emerging Technologies* **114**, 189–204 (2020) <https://doi.org/10.1016/j.trc.2020.02.013>
- [43] Huang, Z., Li, K., Wang, S., Jia, Z., Zhu, W., Mehrotra, S.: Graph structure prompt learning: A novel methodology to improve performance of graph neural networks. *Applied Intelligence* **55**(17), 1113 (2025) <https://doi.org/10.1007/s10489-025-06952-x>